

Kubernetes Mutating Webhook for Pods

<https://gemini.google.com/app/911de51cf43859a9>

User prompt: I have Azure Kubernetes Cluster I need Kubernetes operator which will invoke REST API with dummy payload, get response abck and generate random guid and add as environment variable in pod deployment. This operator should get invoked for all namespaces except system namespaces. Provide complete solution including deployment steps.

Response: To achieve this in Kubernetes, you don't actually need a traditional "Operator" (which reconciles state *after* resources are created). Because Pods are immutable once created, injecting an environment variable dynamically at creation time is best done using a **Mutating Admission Webhook**.

A Mutating Webhook intercepts the Pod creation request in-flight, runs your custom logic (invoking the REST API and generating the GUID), and patches the Pod specification with the new environment variable before it is saved to the cluster.

Here is the complete, end-to-end solution using Python, Flask, and native Kubernetes configurations.

Step 1: Write the Webhook Logic

Create a directory for your project and add the following Python file. This script receives the Kubernetes admission request, makes the REST API call, generates the GUID, and creates a JSON patch.

app.py

```
from flask import Flask, request, jsonify
import uuid
import requests
import base64
import json

app = Flask(__name__)

@app.route('/mutate', methods=['POST'])
def mutate():
    req_data = request.get_json()

    # 1. Invoke REST API with dummy payload
```

```
try:
    # Replace with your actual REST API endpoint
    requests.post("https://httpbin.org/post", json={"dummy": "payload"}, timeout=3)
except Exception as e:
    print(f"API call failed: {e}. Proceeding anyway...")

# 2. Generate random GUID
new_guid = str(uuid.uuid4())

# 3. Create JSON Patch to inject env var
req = req_data.get('request', {})
pod = req.get('object', {})

patch = []
containers = pod.get('spec', {}).get('containers', [])

for i, container in enumerate(containers):
    new_env = {"name": "INJECTED_API_GUID", "value": new_guid}
    if 'env' not in container:
        # If the container has no env array yet, create it
        patch.append({"op": "add", "path": f"/spec/containers/{i}/env", "value": [new_env]})
    else:
        # If it does, append to it
        patch.append({"op": "add", "path": f"/spec/containers/{i}/env/-", "value": new_env})

# Kubernetes requires the patch to be base64 encoded
patch_b64 = base64.b64encode(json.dumps(patch).encode('utf-8')).decode('utf-8')

# 4. Construct the AdmissionResponse
admission_response = {
    "uid": req.get("uid"),
    "allowed": True,
    "patchType": "JSONPatch",
    "patch": patch_b64
}
```

```

    return jsonify({
        "apiVersion": "admission.k8s.io/v1",
        "kind": "AdmissionReview",
        "response": admission_response
    })

if __name__ == '__main__':
    # Webhooks MUST run over HTTPS. Certificates will be mounted via Kubernetes.
    app.run(host='0.0.0.0', port=8443, ssl_context=('/certs/tls.crt', '/certs/tls.key'))

```

requirements.txt

```

Flask==3.0.0
requests==2.31.0

```

Dockerfile

```

FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]

```

Build and push this image to your Azure Container Registry (ACR) or DockerHub:

```

docker build -t yourregistry.azurecr.io/guid-injector:v1 .
docker push yourregistry.azurecr.io/guid-injector:v1

```

Step 2: Generate TLS Certificates

Kubernetes API servers will only communicate with webhooks over HTTPS. We need to generate a self-signed certificate and store it as a Kubernetes Secret.

Run this bash script in your terminal to generate the certs and the secret:

```
# Set namespace for the webhook
NAMESPACE="webhook-system"
kubectl create namespace $NAMESPACE

# Generate keys and certificates
openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key ca.key -subj "/CN=GuidInjectorCA" -days 3650 -out ca.crt
openssl genrsa -out tls.key 2048
openssl req -new -key tls.key -subj "/CN=guid-injector-svc.${NAMESPACE}.svc" -out tls.csr

# Create config for the extension
cat <<EOF > extfile.cnf
subjectAltName = DNS:guid-injector-svc.${NAMESPACE}.svc,DNS:guid-injector-svc.${NAMESPACE}.svc.cluster.local
EOF

# Sign the certificate
openssl x509 -req -in tls.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out tls.crt -days 365 -extfile extfile.cnf

# Create the Kubernetes Secret to hold the certs
kubectl create secret tls guid-injector-certs \
  --cert=tls.crt \
  --key=tls.key \
  --namespace=$NAMESPACE

# Save the CA bundle base64 string (we need this for step 3)
CA_BUNDLE=$(cat ca.crt | base64 | tr -d '\n')
echo "Your CA Bundle is: $CA_BUNDLE"
```

Step 3: Deploy to Azure Kubernetes Service

Now, we will deploy the Webhook server and configure the cluster to route Pod creation requests to it, while ignoring system namespaces.

Create a file named `webhook-deployment.yaml` and replace `yournregistry.azurecr.io/...` with your actual image path.

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: guid-injector
  namespace: webhook-system
spec:
  replicas: 1
  selector:
    matchLabels:
      app: guid-injector
  template:
    metadata:
      labels:
        app: guid-injector
    spec:
      containers:
        - name: injector
          image: yourregistry.azurecr.io/guid-injector:v1
          ports:
            - containerPort: 8443
          volumeMounts:
            - name: certs
              mountPath: "/certs"
              readOnly: true
      volumes:
        - name: certs
          secret:
            secretName: guid-injector-certs
---
apiVersion: v1
kind: Service
metadata:
  name: guid-injector-svc
  namespace: webhook-system
spec:
  ports:
    - port: 443
```

```
targetPort: 8443
selector:
  app: guid-injector
```

Create a file named **webhook-config.yaml**. **Important:** Replace `${CA_BUNDLE}` with the base64 string printed at the end of Step 2.

```
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: guid-injector-webhook
webhooks:
- name: guid-injector.example.com
  clientConfig:
    service:
      name: guid-injector-svc
      namespace: webhook-system
      path: "/mutate"
    # Replace this with your actual base64 encoded CA bundle
    caBundle: "${CA_BUNDLE}"
  rules:
    - operations: ["CREATE"]
      apiGroups: [""]
      apiVersions: ["v1"]
      resources: ["pods"]
    # If the webhook fails/is down, Ignore means pods will still deploy (without the env var)
    # Change to "Fail" if the GUID is strictly mandatory for the app to function.
  failurePolicy: Ignore
    # This ensures the webhook triggers for ALL namespaces EXCEPT the system ones
  namespaceSelector:
    matchExpressions:
      - key: kubernetes.io/metadata.name
        operator: NotIn
        values: ["kube-system", "kube-public", "kube-node-lease", "webhook-system"]
  sideEffects: None
  admissionReviewVersions: ["v1", "v1beta1"]
```

Apply both files:

```
kubectl apply -f webhook-deployment.yaml  
kubectl apply -f webhook-config.yaml
```

Step 4: Test and Verify

1. **Deploy a test Pod** in the default namespace (which is not excluded):

```
kubectl run test-pod --image=nginx
```

2. **Verify the Environment Variable** was injected successfully:

```
kubectl get pod test-pod -o jsonpath='{.spec.containers[0].env}'
```

You should see output similar to this, confirming your API-triggered GUID was injected:

```
[{"name": "INJECTED_API_GUID", "value": "5a6b107e-9764-4bf8-b2a0-410f924e24eb"}]
```

3. **Verify System Namespaces are ignored:**

```
kubectl run system-test --image=nginx -n kube-system  
kubectl get pod system-test -n kube-system -o jsonpath='{.spec.containers[0].env}'
```

This should return nothing, proving the namespaceSelector is successfully protecting your system components.
